

Unlimited Vector Processing for Wireless Baseband Based on RISC-V Extension

Limin Jiang, Yi Shi, Yihao Shen, Shan Cao, Zhiyuan Jiang, and Sheng Zhou

Abstract—Wireless baseband processing (WBP) serves as an ideal scenario for utilizing vector processing, which excels in managing data-parallel operations due to its parallel structure. However, conventional vector architectures face certain constraints such as limited vector register sizes, reliance on power-of-two vector length multipliers, and vector permutation capabilities tied to specific architectures. To address these challenges, we have introduced an instruction set extension (ISE) based on RISC-V known as unlimited vector processing (UVP). This extension enhances both the flexibility and efficiency of vector computations. UVP employs a novel programming model that supports non-power-of-two register groupings and hardware strip-mining, thus enabling smooth handling of vectors of varying lengths while reducing the software strip-mining burden. Vector instructions are categorized into symmetric and asymmetric classes, complemented by specialized load/store strategies to optimize execution. Moreover, we present a hardware implementation of UVP featuring sophisticated hazard detection mechanisms, optimized pipelines for symmetric tasks such as fixed-point multiplication and division, and a robust permutation engine for effective asymmetric operations. Comprehensive evaluations demonstrate that UVP significantly enhances performance, achieving up to $3.0\times$ and $2.1\times$ speedups in matrix multiplication and fast Fourier transform (FFT) tasks, respectively, when measured against lane-based vector architectures. Our synthesized RTL for a 16-lane configuration using SMIC 40nm technology spans 0.94 mm^2 and achieves an area efficiency of 21.2 GOPS/mm^2 .

Index Terms—Vector processor, strip-mining, RISC-V, programming model, hardware implementation.

I. INTRODUCTION

VECTOR processing has been a cornerstone of high-performance computing for decades, offering a specialized approach to handling data-parallel tasks efficiently. By employing a single-instruction-multiple-data (SIMD) technique, vector processing can dramatically improve performance in applications such as scientific computing [1], image processing [2], and machine learning [3]. These advantages are enabled by vector processors that utilize vector registers to execute operations on arrays of data in parallel. This paradigm

leverages data-level parallelism to achieve higher throughput compared to scalar processing.

A notable advancement in the field is the RISC-V Vector “V” (RVV) extension [4], which introduces a flexible and extensible approach to vector processing. Unlike traditional fixed-width vector architectures, the RVV allows dynamic vector lengths (VLs), enabling a more scalable and adaptable execution model. One of its key innovations is register grouping (RG), also referred to as vector length multipliers (LMULs). This mechanism enables the combination of multiple vector registers to form larger logical registers, allowing efficient operations on wider vectors. Conversely, it also supports splitting a register into smaller segments for finer-grained operations on narrow data. By accommodating various data widths dynamically, register grouping ensures efficient resource utilization across diverse workloads. Combined with its extensive set of arithmetic, logical, and data manipulation instructions, the RVV represents a significant leap in flexibility and capability. Moreover, as an open standard, it fosters widespread adoption and drives innovation within the computational community.

Despite its advantages, vector processing faces several challenges that limit its efficiency and applicability. One fundamental drawback of traditional vector architectures is the size constraint of vector registers, which determines the maximum data width that can be processed in a single operation. This limitation necessitates fragmenting larger datasets into smaller segments, incurring control overhead due to additional branch instructions and loop iterations. Such control dependencies can degrade performance, especially for workloads with irregular or large data sizes. Additionally, power-of-two constraints in some architectures, including the RVV’s reliance on power-of-two LMULs, can lead to under-utilization of hardware resources. Workloads requiring non-power-of-two vector lengths may experience wasted computation and energy inefficiencies, as hardware resources are left idle or misaligned with data requirements.

Another challenge lies in the implementation of versatile vector permutations, which are operations that rearrange data within vectors. Such permutations are frequently required in domains like cryptography [5], [6] and wireless signal processing [7]. However, the ability to perform arbitrary permutations efficiently is implementation-specific, varying across architectures. This lack of uniformity creates complexity in developing portable and optimized software solutions.

Just as importantly, wireless baseband processing (WBP) demands a more domain-specific architecture [8]. The computation tasks involved in WBP can be broadly categorized into two types. On one hand, tasks such as channel equal-

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grants 62271300 and 12141107, in part by the Shanghai Municipal Science and Technology Commission under grants 24DP1501100 and 24DP1500600. The corresponding authors are Shan Cao and Zhiyuan Jiang.

L. Jiang, Y. Shi, Y. Shen, S. Cao, and Z. Jiang are with Shanghai Key Laboratory of Chips and Systems for Intelligent Connected Vehicle and School of Communication and Information Engineering, Shanghai University, Shanghai 200444, China. E-mails: {jianglimin, yishi1996, shenyihao, cshan, jiangzhiyuan}@shu.edu.cn

S. Zhou is with Beijing National Research Center for Information Science and Technology, Department of Electronic Engineering, Tsinghua University, Beijing 100084, China. E-mail: sheng.zhou@tsinghua.edu.cn

ization [9] and demodulation [10] are computation-intensive, requiring successive arithmetic operations. On the other hand, tasks like rate-matching [11] involve irregular VLS and permutations, for which mainstream architectures offer limited optimization. Moreover, WBP exhibits a high tolerance for quantization error [12], enabling signal processing with fixed-point arithmetic. This allows the large silicon area typically allocated to floating-point units to be repurposed for more domain-specific fixed-point datapaths tailored to WBP, such as complex number computations and saturation logic, which will be discussed in the following sections. The question now is how to design a reasonable instruction set extension for WBP.

To address these challenges, we propose unlimited vector processing (UVP), a novel approach that removes the dependence on fixed vector register sizes and enables seamless handling of arbitrarily long vectors. This paradigm also standardizes vector permutations to improve portability and performance across diverse computational fields. We propose an unlimited vector programming model to RISC-V instruction set extension (ISE), *Xuvp*, based on insights from software programming. The programming model is implemented at register transfer level (RTL), and we detail the corresponding hardware design. We also analyze the bottlenecks of existing vector processing approaches using benchmark kernels commonly employed in WBP. The key contributions of our work are as follows:

- *Formalization of UVP*: We present the ISE encoding as well as programming model for UVP. In essence, UVP supports non-power-of-two register grouping and leverages larger physical register files to handle vectors of arbitrary length. This model reduces strip-mining instructions, improving efficiency for large data sets. To further enhance flexibility, we categorize vector instructions into two groups: symmetric instructions, where source and destination operands have the same length, and asymmetric instructions, which handle operands with differing lengths. We detail distinct load/store approaches tailored for each category and demonstrate the benefits of this model through two frequently used computational kernels.
- *Hardware implementation of UVP*: We design hardware to support the UVP programming model with a new hazard detection logic optimized for the register grouping strategy. For symmetric instructions, we enhance the data pipeline to efficiently handle complex multiplication and division for fixed-point computation. For asymmetric instructions, we introduce a universal permutation engine capable of shuffling any elements to any position, accommodating arbitrary vector lengths. This hardware design ensures seamless execution of the UVP model, maximizing performance and resource utilization.
- *Evaluation of UVP*: We comprehensively evaluate UVP to demonstrate its performance and efficiency. First, we compare two frequently used kernels with a lane-based vector processing architecture, achieving up to $3.0\times$ and $2.1\times$ performance improvements, respectively. Next, we

synthesize and implement the UVP RTL design and compare it with state-of-the-art architectures, showing superior normalized integer area efficiency. We also analyze UVP performance across different configurations, highlighting its adaptability to varying workloads.

The remainder of this paper is organized as follows. Section II reviews and categorizes existing vector processing methodologies, introducing the fundamental strip-mining approach across different instruction set architectures (ISAs). In Section III, we introduce the design principles of UVP. Section IV details the implementation techniques used to address vector length constraints and permutation challenges. In Section V, we evaluate the performance of the proposed system through benchmarks and case studies. Finally, Section VI summarizes the conclusions of the paper.

II. RELATED WORK AND BACKGROUND

A. Existing Vector Processing Techniques

1) *RVV-based Vector Processors*: A significant body of work focuses on RVV-based vector processors. Both industrial and academic designs have adopted RVV for various applications, from embedded systems to high-performance computing, due to its simplicity and open-source nature. In industry, T-Head and SiFive have developed their own RVV-compatible cores, named XuanTie 910 [13] and X280 [14], respectively. Both vector function units are supported by Linux-ready out-of-order cores. However, at the micro-architecture level, XuanTie 910 [13] integrates the vector function unit directly into the core pipeline, while X280 [14] uses a rather decoupled approach with a dedicated private interface. Additionally, AndesCore supports additional data formats beyond the RVV-specified ones [15], and Semidynamics introduces a fully customizable vector processing unit [16].

In academia, the Ara vector processor continues to evolve in alignment with the RVV specification [17]–[19] and is increasingly becoming a blueprint for other works [20]–[22]. Data-level parallelism is achieved through scalable *lanes*, with the register files divided into banks to acquire higher frequencies and reduce read/write conflicts. In contrast to lane-based designs, other RVV-compliant works implement monolithic register files, allowing each execution unit to access all vector elements within the register files [23], [24]. However, while the ISA strives to address diverse domain requirements, it may fall short in specific scenarios such as time-critical processing in wireless baseband and handling massive data loads in AI applications.

2) *Customizations for Domain-Specific Computations*: In this approach, researchers define specialized instructions on the RISC-V Base ISA, extending it to meet the needs of specific computation tasks. This customization allows for highly efficient hardware implementations tailored to particular workloads. Non-standardized vector processing accelerates data processing in specific domains by customizing critical instructions and implementing dedicated hardware logic for domain-specific tasks [25], [26]. In edge artificial intelligence (AI) applications, processors are often small to conserve power, and models for inference use low-precision data formats to reduce

memory usage. Under these constraints, vector processing in edge AI processors is typically designed in a packed-SIMD style, with additional arithmetic logic units (ALUs) and control logic that enable parallel computation of low-precision operands within a scalar register [27], [28]. For more demanding applications, such as convolutional neural networks (CNNs), additional execution units, like tensor units or systolic arrays, are incorporated to boost throughput [29], [30]. Similarly, in wireless communications, the complexity of massive-input massive-output (MIMO) systems scales exponentially with the number of antennas, leading to the development of vector processors specialized for matrix decomposition [31].

Apart from the need at the ISA-level, customization can also be applied at the micro-architectural level. Both RISC-V² [32] and Lazo et al. [33] emphasize register file organization, implementing dedicated hardware for remapping between the logical vector register allocated by the compiler and the physical register specified by the hardware implementation. While this alleviates the complexity of register allocation algorithms for the compiler, it incurs not only increased chip area but also additional execution clock cycles per instruction at the hardware level. Furthermore, UVE [34], [35] presents data streaming support and an emulation platform. These modifications are based on existing scalar core architectures to support various memory access patterns. Nonetheless, data streaming with indirect patterns suffers from complicated descriptors and limited parallelism.

3) *Leveraging Scalar Cores for Vector Processing*: In contrast to dedicated vector processors, some research has focused on modifying commercial-off-the-shelf scalar cores to function as vector processors. Multiple scalar cores are used in parallel, where each core processes a part of the vector in lockstep, with the instruction fetch pipeline of most cores disabled. Essentially, the scalar cores operate together to mimic vector processing by distributing vector elements across the cores and executing the same instruction on multiple data elements simultaneously. This approach is often considered an extension or additional feature of manycore architectures [36]–[38], where a set of scalar cores is utilized to handle vector workloads, rather than relying on specialized vector units.

While this approach can leverage existing scalar cores to perform vector-like operations, it tends to have lower computational efficiency compared to dedicated vector processors. This is because, although the scalar cores are working in parallel, the underlying hardware was not initially designed to handle vectorized tasks. As a result, there is often significant overhead from logic that is only useful for scalar processing, such as complex control logic and instruction fetch mechanisms that need to be reconfigured for lockstep execution. Furthermore, this approach lacks the dedicated vector units and optimizations found in true vector processors, leading to suboptimal performance in terms of throughput and energy efficiency.

4) *Digital Signal Processors for WBP*: Very long instruction word (VLIW) architectures have been widely adopted in digital signal processors (DSP) for their ability to exploit instruction-level parallelism (ILP) through static scheduling. In VLIW processors, multiple operations – including scalar, vector, and even tensor [39] instructions – are bundled and

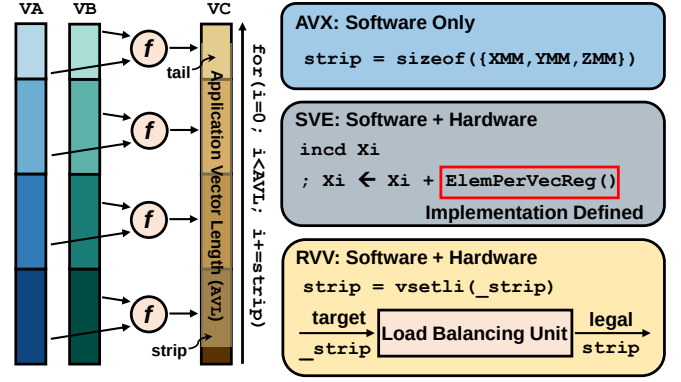


Fig. 1. The concept of strip-mining and its implementation in mainstream ISAs.

issued simultaneously in a single long instruction word by the compiler [40], [41]. The hardware is greatly simplified compared to superscalar or vector designs, since scheduling, hazard avoidance, and resource allocation are all handled by the compiler. This makes VLIW especially attractive in domain-specific scenarios with predictable workloads and tight power constraints, namely computer vision tasks [42] and WBP [43], [44].

While VLIW DSPs offer high efficiency under regular and static analyzable workloads, they suffer from several limitations. First, the burden of complex scheduling and register allocation falls entirely on the compiler, which demands deep expertise in the target architecture to achieve optimal performance – thereby reducing portability and increasing development effort. Second, static instruction bundling lacks flexibility in handling dynamic workloads or irregular data patterns, often resulting in suboptimal hardware utilization.

B. Vector Strip-mining

One of the key techniques in vector processing is strip-mining, which is used to handle data exceeding the capacity of a vector register. Fig. 1 illustrates the concept of strip-mining. Essentially, strip-mining divides the data into smaller *strips* that fit within the available vector register length. These strips are processed sequentially in a loop until all the data is covered. For data sizes that are not a multiple of the vector register length, additional *tail* processing is required. Given two source vectors, VA and VB, with a vector length of AVL, a complete strip-mining procedure for an operation f to compute VC involves first calculating $T = \lfloor \frac{AVL}{strip} \rfloor$ iterations of parallel computation fully utilizing the hardware. Subsequently, an additional computation is performed for the $AVL(1 - strip \cdot T)$ elements that remain.

Fig. 1 also illustrates three mainstream implementations of vector processing, highlighting their approaches to handling strip increments. In AVX programming, the strip size is fixed to the capacity of vector registers in the micro-architecture, such as XMM, YMM, or ZMM. Here, hardware does not participate in strip-mining control; instead, programmers calculate the strip size manually, often using the `sizeof` function. Both SVE and RVV adopt vector-length agnostic approaches, requiring hardware feedback to report the current micro-architectural attributes. SVE uses the `incd` assembly to incre-

TABLE I
INSTRUCTION ENCODING FOR UVP

Bit field	Name	Description
[31:25]	funct7[6:0]	Operations under the category.
[24:23]	vew[1:0]	Element width of the vector.
[22]	vmask	Whether to read predicate registers before computation.
[63:52], [21]	vd_head[12:0]	Starting number of destination vector register.
[51:42], [20:18]	vs2_head[12:0]	Starting number of the second source vector register.
[41:32], [17:15]	vs1_head[12:0]	Starting number of the first source vector register or scalar register number.
[14:12]	funct3[2:0]	Categories of vector instructions.
[11:7]	rs_avl[4:0]	Scalar register number storing AVL.
[6:0]	opcode[6:0]	Operation code of current instruction.

ment a counter based on the register size, which can vary by hardware implementation, typically ranging from 128 to 2048 bits. Combined with the `whilelt` and branch assemblies with predication, strip-mining loops can be implemented through compiler auto-vectorization, eliminating the need for detailed micro-architectural knowledge. Additionally, RVV enforces VL constraints via a load balancing unit controlled by the `vsetvli` instruction. Programmers specify a desired level of parallelism, and then the load balancing unit adjusts it to suit the current architecture. Finally, the adjusted strip size is used to update the strip counter.

Strip-mining arises as a trade-off among data parallelism, hardware utilization, and software feasibility. However, when applied to long vector processing, it encounters two primary challenges. Firstly, the strip size is constrained by the capacity of vector registers. Although RVV employs power-of-two register group multipliers to mitigate this limitation, the overhead of strip-mining – such as increment/decrement, comparisons, and branching – is not negligible in long vector scenarios. Furthermore, an improper strip size can lead to fragmented memory access patterns. Secondly, vector processing is limited by the number of available vector registers, especially when configuring a bloated LMUL. When multiple operations are bundled within a strip-mining loop, frequent register spilling and filling can occur. These two challenges not only increase memory traffic but also degrade overall throughput, further exacerbating inefficiencies in long vector processing. Compared to conventional vector programming, UVP leverages its support for non-power-of-two RG, effectively alleviating the issues of register under-utilization and excessive strip-mining instructions. Even when the VL is a power of two, UVP still demonstrates advantages due to reduced instruction fetch overhead and fewer pipeline stalls.

III. UNLIMITED VECTOR PROCESSING

The formalization of the proposed ISE is divided into three subsections. First, we present the instruction encoding of the ISE, and introduce new instructions tailored to the unique demands of WBP. Next, we describe our new programming model and the organization of RG. Finally, we illustrate the practicality of our approach through two representative kernels in WBP, showcasing the effectiveness of the ISE and programming model.

A. ISE Specifications

Table I presents one possible instruction format of UVP. The UVP instruction bit width is 64 bits to accommodate the expanded physical vector register files (VRFs). While the instruction fetch may require additional clock cycles, this overhead can be offset by the SIMD nature. Details of the instruction formats are as follows.

1) *Operation Code*: The UVP ISE is designed based on the RISC-V ISA, which provides custom operation codes that allow developers to add new instructions while maintaining compatibility with the standard set. Our ISE utilizes *custom-1* and *custom-2* for this purpose.

2) *AVL Register*: A general-purpose scalar register is used to store the VL, with the register number encoded in the instruction. Unlike `vsetvli`, we simply reuse the `add` or `addi` instructions to inform hardware of the AVL for the entire vector. For example, prior to executing a `uvp_add` instruction with `a0` as the AVL register, the compiler emits configuration assembly as shown below:

```
addi a0, a0, <immediates>
uvp_add v0, v0, v1, a0. (1)
```

Here, `v*` represents vector registers. Furthermore, the compiler can also optimize the live interval of scalar registers when the VL remains constant or is modified by basic operations across instructions.

3) *Instruction Categories*: The instructions can be categorized based on the types of source and destination vectors. For source operands, one operand can be either a vector or a scalar. For destination operands, results can be written to either VRFs or dedicated predication registers.

4) *Predication Registers*: For hardware efficiency, the ISE only supports one predication register. This design eliminates the need for complicated data widening and narrowing logic in the data pipeline, which can arise from bit-width differences between vector and mask elements. The predication register length matches the total number of bytes that all VRFs can process.

5) *Vector Element Width*: The ISE currently supports only `char` and `short` data types, as their bit widths are sufficient for quantization in WBP. Silicon previously allocated to less frequently used units, such as floating-point units and high

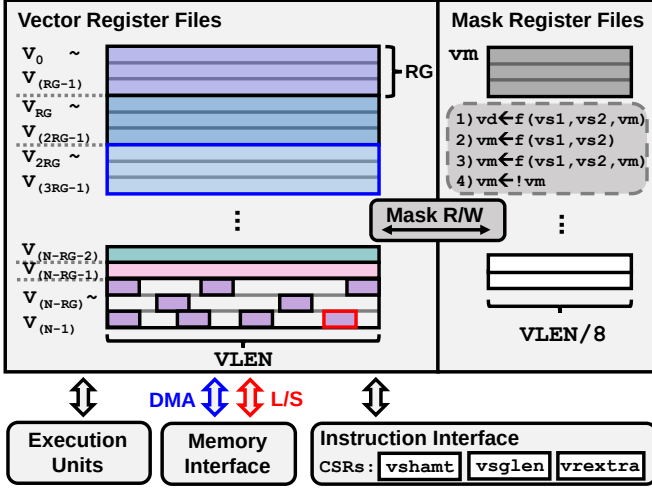


Fig. 2. The programming model of our proposed unlimited vector processing (VLEN: Vector length of a single vector register).

bit-width multipliers, can be redirected to enhance data parallelism.

6) *Supported Operations*: In addition to basic fixed-point arithmetic, mask and reduction instructions, the proposed ISE allows different VLs between source and destination vectors through two new instructions: `uvp_gather` and `uvp_scatter`. Unlike `vrgather` in RVV, which operates under the same VL and can lead to low VRF utilization in certain LMUL configurations, these *asymmetric* instructions maximize VRF efficiency when there is a substantial VL difference between source and destination registers. For instance, scattering often results in longer vectors:

$$vd[vs2[i]] \leftarrow vs1[i]. \quad (2)$$

The `uvp_scatter` instruction eliminates the need to pad vector registers associated with `vs1` and `vs2` to match the VL of `vd`. Conversely, gathering often results in shorter vectors:

$$vd[i] \leftarrow vs1[vs2[i]]. \quad (3)$$

In this case, registers related to `vd` and `vs2` are cropped to a shorter VL.

7) *Vector Registers*: For hardware implementation, single-port static RAMs (SRAMs) can be used instead of registers or register files due to their advantages: (i) SRAMs offer higher cell density, saving silicon area when the number of physical registers is large; and (ii) it is unlikely that more than two read/write requests will occur on an SRAM in a single clock cycle. In case of burst requests, implementing a request queue with back-pressure is more efficient. The depth of VRFs is up to 2^{13} as specified in Table I, but it can be further expanded by using `uvp_vsetcsr` instruction to write extra bits into control and status registers (CSRs).

B. Programming Model

To address the issue outlined in Section II-B, we propose a programming model for UVP that is tailored for high-throughput vector processing. UVP is achieved by addressing two key constraints: the granularity of vector register grouping and the hardware strip-mining. Unlike traditional ISAs, UVP

TABLE II
RG ORGANIZATION OF ASYMMETRIC INSTRUCTIONS

	vd	vs1	vs2
<code>uvp_gather</code>	V_{N-RG-2}	$V_{N-RG} \sim V_{N-1}$	V_{N-RG-1}
<code>uvp_scatter</code>	$V_{N-RG} \sim V_{N-1}$	V_{N-RG-2}	V_{N-RG-1}

offers greater flexibility in vector register grouping, allowing for non-power-of-two groupings. This enables programmers to access arbitrary consecutive vector registers within a single instruction, thereby reducing the execution overhead of strip-mining loops. Additionally, UVP shifts strip-mining from software to hardware by leveraging the AVL register in instruction encoding, which reduces the frequency of memory accesses through finer-grained register management. However, UVP introduces challenges for compiler implementation. While mainstream ISAs exhaust registers and sub-registers with relatively low complexity, UVP's flexibility necessitates more sophisticated handling. Although we have implemented several passes in an LLVM compiler to address these challenges, the details lie beyond the scope of this study and will be explored in future work.

An overview of the proposed programming model is illustrated in Fig. 2. The key features of the programming model are detailed as follows.

1) *Vector Register Files*: The VRF usage of UVP offers greater flexibility, with RGs dynamically adjusted based on different instructions. RG is determined offline by two factors: the starting register number, RG_{head} , allocated by RA algorithms of the compiler, and the vector data type, defined as $RG_{type} = L \cdot VEW$, where L is the VL specified by the programmer through high-level language. At compile time, RGs are allocated by calculating its range from RG_{head} to $RG_{tail} = RG_{head} + RG_{type}/VLEN$. During runtime, the hardware accesses vector registers based on the AVL register specified in the instruction encoding, which allows for cases where the user-defined size exceeds the actual runtime vector size. This design introduces additional complexity in hardware implementation, as UVP requires managing two degrees of freedom (RG_{head} and RG_{type}), in contrast to traditional ISAs, which typically rely on only one (RG_{head}).

RG organization varies depending on the instruction type. Symmetric instructions execute element-wise computations, where the destination registers have the same length as the source registers. Operands in symmetric instructions are densely arranged, making them well-suited for direct memory access (DMA). This dense arrangement allows efficient vector load/store operations, typically performed before or after strip-mining. As illustrated in Fig. 2, the source and destination RG in symmetric instructions occupy the same number of vector registers. In contrast, asymmetric instructions involve differing lengths between source and destination registers. The source operands to be gathered or scattered are sparsely arranged. In such scenarios, scalar load/store instructions from the scalar core are more efficient than vector load/store operations. Table II presents the possible RG organizations for asymmetric instructions, aligned with Fig. 2.

2) *Mask Register Files*: Mask register files (MRFs) store predication results, with a size one-eighth that of the VRF,

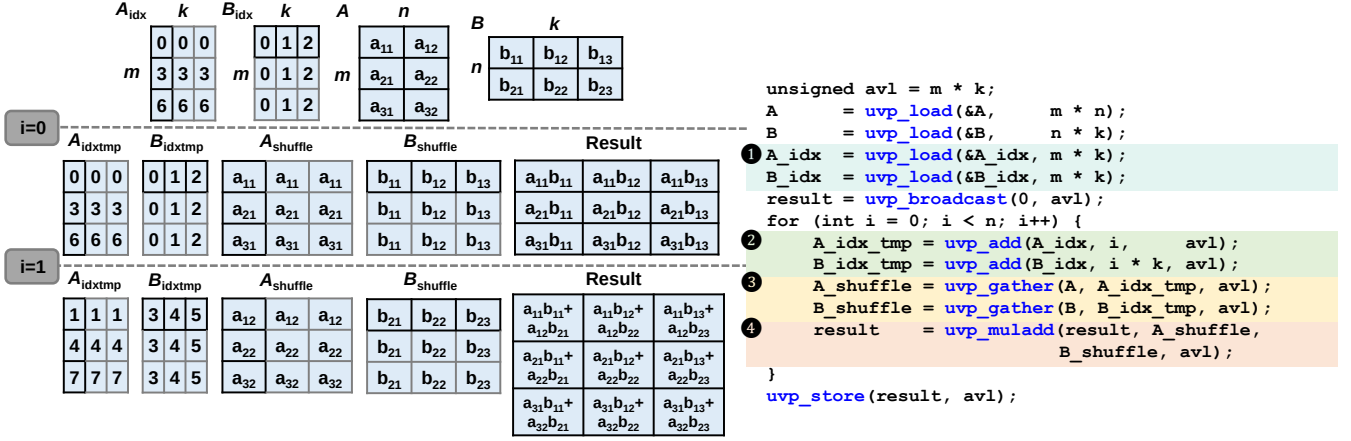


Fig. 3. A *matmul* kernel multiplying a 3×2 matrix by a 2×3 matrix, requiring two iterations.

holding mask content in bits. For implementation simplicity, there is a single mask register, *vm*, which holds the latest predication result. MRF contents are inaccessible via the memory interface and can only be modified by specific instructions. For example, a compare instruction (e.g. *uvp_seq*) can operate with or without input from mask registers, and its result can be written back to either the VRF or MRF. Additionally, the mask register can reverse predications using the *vmnot* instruction, enabling efficient handling of vector if-else conditions.

3) *Control and Status Registers*: Control and status registers (CSR) provide additional information for UVP through a dedicated instruction interface. For instance, *vsglen* specifies the vector length of destination registers in scatter and gather instructions, addressing the asymmetric nature of these instructions. The *vrextra* register extends instruction encoding space to accommodate the indices of vector registers when the current instruction format cannot specify a sufficient number of vector registers. For computation, *vshamt* denotes the shift amount of operands to be applied to operands before or after the computation, a feature which will be further clarified in Section IV-C. Although CSR configuration instructions might be required prior to each vector instruction, the overhead is minimal and easily amortized due to the efficiency of long vector processing.

In summary, we define the syntax of the proposed programming model in the C programming language as follows:

$$[V_{dest} =] f_{uvp1}(V_{src} | Addr, [B_r,] AVL), \quad (4)$$

$$[V_{dest} =] f_{uvp2}(V_{src1}, V_{src2}, [V_{src3}, B_r, B_w,] AVL), \quad (5)$$

$$f_{uvp3}(V_{dest1}, [V_{dest2},] V_{src1}, V_{src2}, [V_{src3}, V_{src4}, B_r,] AVL). \quad (6)$$

Here, the three types of f_{uvp} correspond to load/store operations, normal arithmetic instructions, and complex/asymmetric instructions, respectively. The optional boolean flags B_r and B_w indicate whether the hardware should read from or write to MRFs.

C. Kernel Examples

To showcase the advantages of UVP, we present two kernel examples commonly used in WBP. Leveraging large vector registers and flexible vector organizations, these kernels are

rewritten to achieve greater efficiency and improved performance.

1) *Matrix Multiplication*: Let a_i denote the i -th column vector of the matrix $A_{m \times n}$, and b_j denote the j -th row vector of the matrix $B_{n \times k}$. The resulting matrix $C_{m \times k}$ from their multiplication can be expressed as

$$C = \sum_i \left(\left[\begin{matrix} a_i & a_i & \cdots & a_i \end{matrix} \right] \odot \left[\begin{matrix} b_i \\ b_i \\ \vdots \\ b_i \end{matrix} \right] \right), \quad (7)$$

where $\odot$ is the Hadamard product.

As shown in Fig. 3, matrix multiplication (*matmul*) on UVP can be performed in the following steps. ❶ **Permutation matrix generation**. Two permutation matrices can be generated as follows:

$$\Pi_A = \left[\begin{matrix} \pi_A & \pi_A & \cdots & \pi_A \end{matrix} \right], \quad (8)$$

$$\Pi_B = \left[\begin{matrix} \pi_B & \pi_B & \cdots & \pi_B \end{matrix} \right]^T, \quad (9)$$

where π_A and π_B represent the column vector and row vector of matrices Π_A and Π_B , respectively:

$$\pi_A = [0 \quad n \quad \cdots \quad (m-1)n]^T, \quad (10)$$

$$\pi_B = [0 \quad 1 \quad \cdots \quad k-1]. \quad (11)$$

The permutation matrices can either be stored in advance or generated dynamically by a sequence of fundamental instructions, given their simplicity. ❷ **Permutation matrix update**. Within the calculation loop, the permutation matrices are updated by adding scalar values corresponding to the iteration variables. ❸ **Element shuffling**. Gather instructions are utilized alongside the updated permutation matrices to extract specific elements from the original data matrices. These extracted elements are reorganized to form new matrices aligned for the subsequent computation. ❹ **Resulting matrix accumulation**. At the end of each computation loop, the resulting matrix is updated by accumulating the products of the shuffled matrices. Notably, the permutation matrix and the shuffled matrix in steps ❸ and ❹ can be reduced into a vector

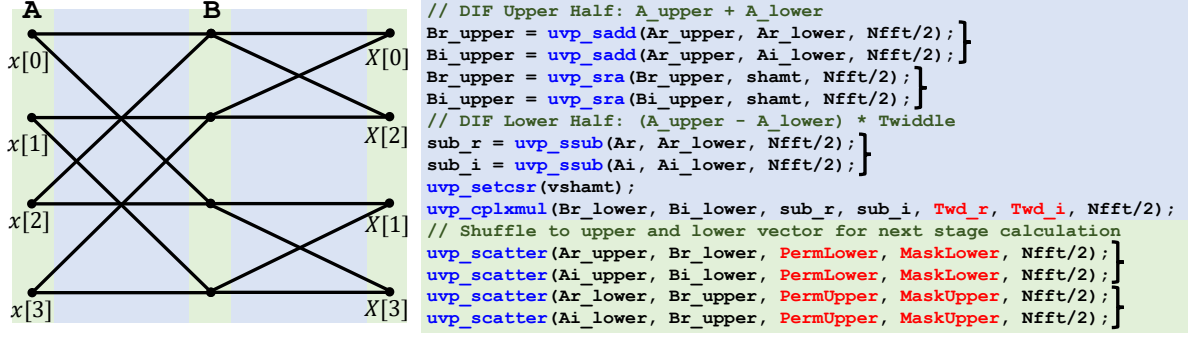


Fig. 4. A DIF fft kernel featuring saturation and complex multiplication instructions within each stage of the dataflow. Variables in red denote prepared permutation vectors for gather and scatter operations.

due to their identical row-wise or column-wise structure. This further reduces the kernel runtime.

In this case, UVP demonstrates a more flexible approach to data computation. The size of the RG is directly influenced by the programmer-specified vector length (`avl`), provided the total number of source and destination vector registers remains within the overall VRF capacity. Compared to traditional *matmul* [45], [46], which has time and space complexities of $\mathcal{O}(N^2)$ and $\mathcal{O}(N)$, respectively, UVP achieves higher throughput by utilizing large VRFs. This optimization effectively reverses the time and space complexity trade-off, significantly enhancing performance in scenarios that require intensive vector operations.

2) *Fast Fourier Transform*: Existing kernel implementations of fast Fourier transform (*fft*) [46] have already optimized the computation process by calculating stage by stage, employing a “shuffle-butterfly-shuffle” strategy within each stage instead of recursively calling a half-point *fft*. However, these implementations face several limitations:

- **Vector Register Size Constraints.** The limited size of existing vector registers restricts the maximum FFT point size. In wireless communication, where FFT size are often large ($N_{fft} \geq 2048$), this becomes a significant bottleneck.
- **Complex Number Representation.** Signal processing relies heavily on data represented as complex numbers, which necessitates efficient complex multiplication. The current implementations lack the ability to reduce instruction counts and efficiently chain the dataflow for such operations.
- **Data Saturation Needs.** Signal processing frequently requires data to saturate to the maximum and minimum values representable in small data types to minimize system error. This demands tailored operations for saturation arithmetic apart from addition and subtraction.

Fig. 4 illustrates a decimation-in-frequency (DIF) *fft* kernel with several enhancements. The expanded VRFs allow for the computation of larger-point FFTs. Scatter instructions, combined with permutation vectors and mask flags (marked in red), are employed to re-organize vectors after the butterfly calculation for the next stage of computation. These vectors can either be pre-stored or calculated at runtime, depending on throughput requirements. The instruction count is further optimized by combining the real and imaginary parts of the

signal into one single vector. During the calculation, a specialized complex multiplication instruction, `uvp_cplxmul`, reduces the instruction count from six (four multiplications and two add/subtract instructions) to a single instruction, streamlining the computation and enhancing dataflow efficiency. Moreover, the `vshamt` register can be pre-configured using the `uvp_setcsr` instruction to handle data overflow during calculations, which eliminates the need for additional overflow management operations.

IV. OVERALL MICROARCHITECTURE

Building upon the proposed programming model, we design an architecture that addresses both hardware strip-mining and WBP requirements. As shown in Fig. 5(a), the microarchitecture is divided into four components. (i) The main sequencer is responsible for routing instructions to the appropriate execution units and stalling the pipeline when there are RG conflicts. (ii) The memory-map-to-lane conversion module contains address generation logic, converting between VRF SRAM addresses and the memory-mapped address organization. A configurable bus interface, compliant with the Advanced eXtensible Interface (AXI), allows the design to access memories actively or passively. (iii) Scalable lanes are dedicated to element-wise computations with the ALUs accessing in-situ VRFs and MRFs. (iv) The element exchange engine (EXE) enables efficient permutations of vector elements across the lanes.

A. Control Path for UVP

The control path of the UVP begins at the main sequencer, traverses the execution units within lanes and the EXE, and concludes at VRFs, as depicted by the blue lines in Fig. 5(a). Unlike traditional strip-mining techniques, where the instruction decoder determines the maximum number of vector elements to process per instruction based on the size of physical registers, vector element type, and the LMUL (if applicable) [17], UVP achieves hardware strip-mining by pipelining the AVL, decoded from the general-purpose register index in the RISC-V ISA, within the main sequencer. Afterwards, the AVL is further divided within the lane sequencer, with the VL for each lane i calculated directly using the pipelined AVL register:

$$VL_i = \lfloor AVL/N_{lane} \rfloor + N_{tail,i}, \quad (12)$$

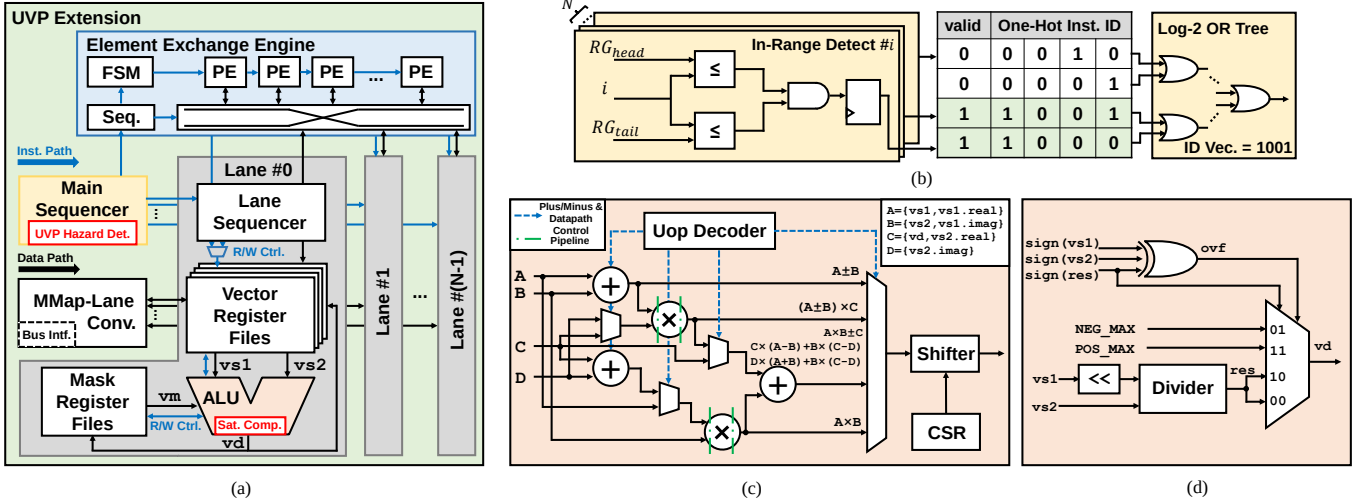


Fig. 5. The overall top view of the UVP microarchitecture and key subdesigns. (a) The overall top view of the UVP microarchitecture, depicting the arrangement and interaction of core modules. (b) Hazard detection scheme of within the main sequencer, focusing on handling vector operand dependencies. (c) Complex arithmetic unit within the ALU, equipped with a final-stage shifter for precise operations. (d) Peripheral logic design surrounding a divider, dedicated to overflow detection and calculation.

where N_{lane} denotes the number of lanes in the UVP extension, and $N_{tail,i}$ is defined as:

$$N_{tail,i} = \begin{cases} 0 & (AVL \bmod N_{lane}) \geq i \\ 1 & (AVL \bmod N_{lane}) < i \end{cases} \quad (13)$$

The calculated result determines the number of micro-operations (uops) each execution unit must perform. This approach eliminates the VL constraint specified in the RVV specification, enabling UVP to directly execute vector processing based on the programmer-defined AVL.

The control signals proceed to fetch operands for both the lanes and the EXE for VL_i iterations. Within each lane, the lane sequencer requests operands from the VRFs and MRFs. The request logic utilizes a multiple-input-single-output arbiter, which handles requests from the in-situ lane as well as the processing elements (PEs) in the EXE. Along with the fetched data, the control signals are further broken down into smaller uops for multi-pipeline execution units. Upon execution, the control signals direct the results back to the VRFs or MRFs. Similarly, for EXE, a finite state machine (FSM) is triggered by the internal sequencer, which itself is activated by the main sequencer. The PEs within the EXE request and write back operands through the same arbiter, ensuring efficient operand management across lanes.

Control signals also facilitate the coordination of lanes and the EXE to execute reduction operations, which are carried out in two stages. In the first stage, intra-lane reduction is performed, where each lane reduces its VL_i elements as part of normal symmetric operations. In the second stage, the EXE handles inter-lane reduction over $\log_2 N_{lane}$ iterations. During iteration n (where $n = 0, 1, \dots, \log_2 N_{lane} - 1$), the EXE fetches operands from lane $(m2^{n+1} + 2^n)$ and writes back to lane $m2^{n+1}$, $m = 0, 1, \dots, N_{lane}/2^{n+1}$. After completing these steps, the final reduction result is written back to the VRFs in lane 0.

B. Hazard Detection Logic for UVP

Another challenge, apart from hardware strip-mining, is hazard detection. Hazards arise when operands are repeatedly accessed in consecutive instructions, requiring hardware to ensure that the subsequent instructions delay any read or write operations until the previous instruction finishes its write-back. Given a total of N physical vector registers, the sum of available logic registers across different LMULs in RVV is calculated as $\sum_{n_{lmul}=0}^3 N/2^{n_{lmul}}$, since only registers $V_{k2^{n_{lmul}}}$, $0 \leq k \leq N/2^{n_{lmul}} - 1$ can serve as the head vector of an RG. However, in UVP, RGs can be arbitrarily assigned, meaning any vectors can act as the head or tail of an RG. This flexibility results in a total of 2^{N-1} possible logic register configurations.

In UVP, hazard detection complexity surpasses that of power-of-two RG configuration. As shown in Fig. 5(b), each physical register is examined to verify whether it falls within the specified range, defined by the head and tail of an RG. Validity information is stored in a table along with a one-hot encoded instruction identity (Inst. ID) assigned by an instruction monitor, which manages up to $N_{ID} = 8$ concurrent instructions. This table feeds into a $\log_2 N$ -level OR tree, using a validity mask to condense information into an ID vector that flags instruction occupation. The ID vector then updates an $N_{ID} \times N_{ID}$ hazard table, which informs subsequent modules of potential pipeline stalls. Validity and IDs clear automatically when the instruction completes. This approach requires $2N$ more comparators than the traditional data hazard recording methods [17], but it enables a more adaptable RG strategy.

C. Computation within Lanes

A lane operates as a subordinate packed-SIMD core, consisting of a lane sequencer that issues uops, temporary storage for operands within mask and vector register files, and an ALU that executes parallel computations. Execution units are further enhanced to support complex multiplication and saturation division, specifically tailored for WBP. As depicted

in Fig. 5(c), a complex arithmetic unit (CAU) includes multiple datapaths designed to handle multiplication-related operations. A uop decoder interprets the instructions and generates control signals for adders and datapath multiplexers. Supported instructions include addition/subtraction, multiplication followed by addition/subtraction, addition/subtraction followed by multiplication, and dedicated complex multiplication. To improve timing performance, two pipeline registers are placed before and after the multipliers. Calculations are performed in full precision by default. A configurable truncate module, managed via `vshamt` CSR, allows operations in fixed-point formats to meet precision and application-specific requirements.

Fix-point division requires careful consideration of two issues: (i) Direct division can lead to underflow, resulting in imprecise number representation and loss of fractional details; and (ii) the quotient may overflow when the dividend is sufficiently small. For a number represented with an m -bit integer part and an n -bit fractional part, noted as mQn , the full-precision calculation of between a divisor of aQb and a dividend of cQd produces a quotient format of $(a + d + 1)Q(b + c)$. To mitigate underflow, an additional left shift of s can be applied to the divisor using `vshamt` CSR. This shift increases the fractional precision of the quotient at the cost of reducing its integer precision, yielding a quotient format of $(a + d + 1 - s)Q(b + c + s)$. As illustrated in 5(d), a configurable left shift logic is implemented before the divisor input. To handle overflow, a detection mechanism is implemented using an XOR gate. The gate takes the sign bits of the divisor, dividend, and unsaturated quotient as inputs. Based on the sign of the unsaturated quotient, the output is saturated to its minimum or maximum value accordingly.

D. Element Exchange Engine

The EXE in the UVP extension enables element gather and scatter operations without requiring memory access. While some vector processing microarchitectures support only limited shuffling of vector elements – often at the granularity of a physical register to keep hardware design simpler – this limitation can lead to frequent memory access as the VL increases. In such cases, vectors must first be stored to memory and then reloaded using memory-based gather/scatter instructions. In contrast, the EXE can gather and scatter elements to any positions within VRFs, facilitating efficient data manipulation across elements without engaging memory.

Highlighted in Fig. 5(a), the EXE consists of an EXE sequencer, an FSM, PEs for data shuffling, and an interconnect. Fig. 6 further details the processing sequence, where the FSM oversees the EXE's operations, prioritizing data integrity over throughput to mitigate hazards compared to a pipelined approach. The FSM, depicted in Fig. 6(a), comprises three main stages: configuration, data movement, and instruction commit. In the configuration stage, the FSM decodes instructions and determines the number of data movements per PE as the EXE sequencer receives a new command. Counter and threshold values are reset and loaded into the PEs. During data movement, each PE loads indices and elements for shuffling and transmits them to the interconnect module, which then

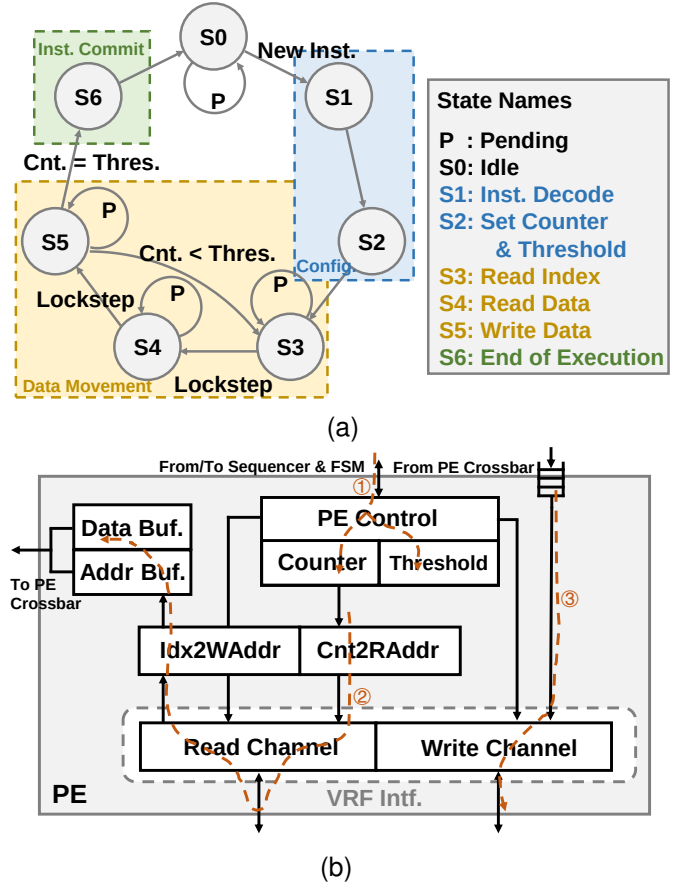


Fig. 6. Details inside EXE. (a) A finite state machine that controls the EXE. (b) Hardware design of shuffle processing elements.

guides the data to the appropriate PEs. While receiving and writing back the shuffled data into the VRF, the FSM ensures all the PEs operate in lockstep before advancing. Finally, in the instruction commit stage, a complete signal is sent to the main sequencer, which updates the instruction monitor and data hazard table.

Based on the FSM design, each PE is implemented to fetch and write back vector elements in a cyclic $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ procedure, as illustrated in Fig. 6(b). A PE control with a counter and a threshold register is included to manage all subsequent modules according to the FSM specifications. The write and read channel interface with vector operands, with operand requests managed by an arbiter within each lane. Additionally, a counter-to-read-address (`Cnt2RAddr`) module is included to sequentially read elements from both the index and target vector, beginning from the first element of an RG. Addressing differs between VRF SRAMs and the index vector contents: VRF SRAMs are accessed within the lane, while index vectors pertain to the RG and span multiple lanes. Therefore, two conversion modules, `Idx2PE` and `Idx2WAddr`, are implemented to direct data to the correct PE and to compute the SRAM address within the corresponding lane, respectively.

V. EXPERIMENT RESULTS

In this section, we evaluate our extension at both the software and hardware levels. At the software level, we

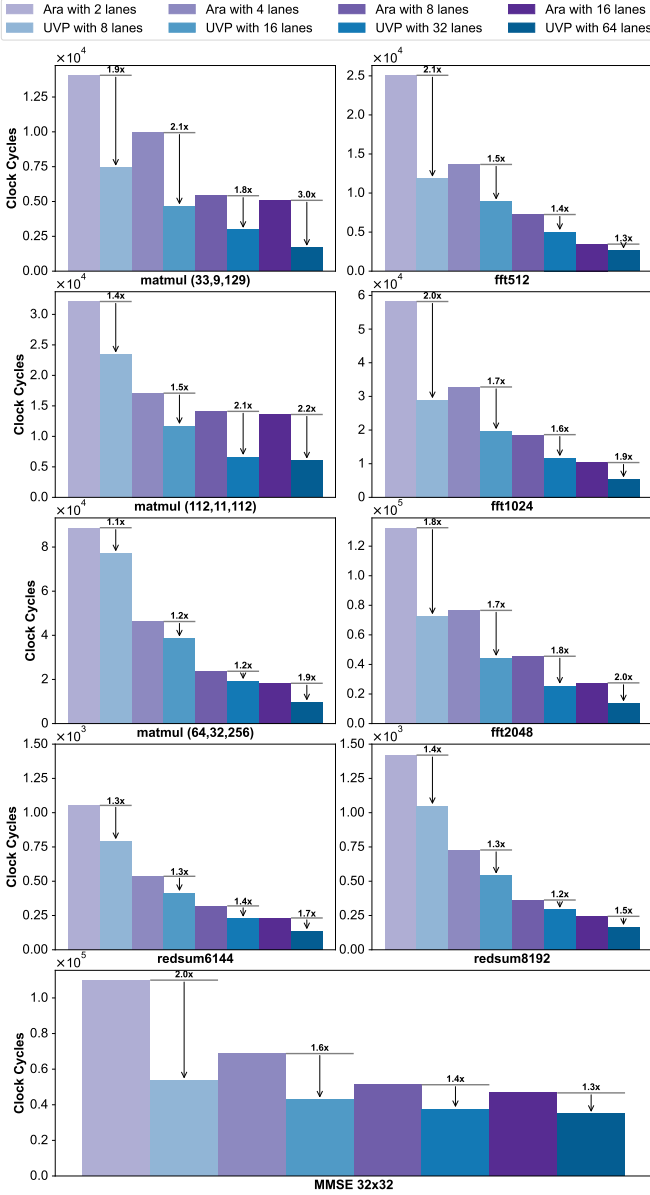


Fig. 7. Comparison of UVP with Ara in the number of clock cycles across various configurations and kernels. Note that we compare the L -lane Ara with the $4L$ -lane UVP (as shown in the legend, with pairs organized in columns) to ensure a fair comparison, given the equal number of execution units.

compare kernels and instruction counts introduced in Section III-C with the open-source RVV processor Ara [17] under various configurations. At the hardware level, we compare our proposed architecture with other state-of-the-art vector processors, providing a detailed architectural breakdown to highlight specific design benefits.

A. Kernel Comparison

We select *matmul*, *fft*, and *redsum* kernels from [46] as benchmarks to compare with our design. In addition, a minimum mean square error (MMSE) kernel was implemented to compare with [47] in multiple-input, multiple-output (MIMO) systems. To align with the execution unit configurations, we modified the element type of the vectors to `int16` in each kernel. At the hardware level, the number of lanes in UVP is set to be four times that of Ara, due to the wider SRAMs and

the increased DLP of Ara. Clock cycles are measured using Synopsys VCS and QuestaSim, respectively. Fig. 7 shows the kernel speedup across four lane configurations. *matmul* (m, n, k) represents an $A^{m \times n} \times B^{n \times k}$ matrix multiplication, *fftN* represents a N -point FFT, and *redsumM* indicates a reduction summation over M elements. For MMSE estimation, we adopt a 32×32 window size, consistent with [47].

1) *Matrix Multiplication*: The speedup for *matmul* ranges from $1.1\times$ to $3.0\times$, with the greatest improvements observed when the matrix dimensions are not powers of two, achieving at least $1.4\times$ speedup. In the proposed *matmul* kernel example, RGs under different AVL ($m \times k$) configurations are more compact compared to the power-of-two RGs. This results in fewer strip-mining loops and switches, as well as register vacancies. An additional source of speedup lies in the loop iteration strategies: Ara's kernel operates at a complexity of $\mathcal{O}(m \times k / b^2)$, while the UVP kernel achieves $\mathcal{O}(n)$ by trading off increased space complexity to store matrix elements. Here, b denotes the block size used to configure `vsetvli` instructions.

2) *Fast Fourier Transform*: For the *fft* kernel, flexible RGs and larger VRFs enable UVP to achieve a performance improvement ranging from $1.3\times$ to $2.1\times$. The performance gap becomes more significant when the input data for the butterfly computation in a stage exceeds the capacity of vector registers in Ara. In such cases, a potential solution would be to call the most suitable kernel for the current hardware configuration and recursively calculate the FFT for larger points. However, this divide-and-conquer approach would introduce additional memory accesses. Under UVP, more data can be loaded into registers and grouped beyond the RVV specification, mitigating the need for excessive memory accesses and strip-mining loops. Additionally, the inclusion of complex multiplication instructions further reduces the clock cycles, contributing to overall improved performance.

3) *Reduction Summation*: Reduction operations are widely used in applications such as low-density parity-check (LDPC) decoding and signal correlation, and continue to be the subject of performance optimization efforts [48], [49]. The fourth row of Fig. 7 shows the performance of reduction summation for both power-of-two and non-power-of-two input lengths. Despite having four times the number of lanes compared to Ara – requiring two additional stages of element permutation – UVP consistently achieves at least a $1.2\times$ speedup. This improvement is attributed to less strip-mining and configuration instructions, as well as the elimination of scalar reductions across vector instructions.

4) *MMSE Estimation*: The complexity of matrix inversion has prompted extensive optimization efforts across platforms [50], [51], with Cholesky decomposition being the most commonly adopted method. The bottom row of Fig. 7 compares the performance of our implementation with that of [47], using a 32×32 matrix window for a 2×2 MIMO system. To avoid under-utilization of the VRF, multiple MMSE kernels are executed in parallel under larger lane configurations. UVP still outperforms the Ara implementation, owing to its flexible element reorganization for matrix operations and enhanced support for complex multiplications.

TABLE III
IMPLEMENTATION RESULTS OF THE PROPOSED VECTOR EXTENSION COMPARED WITH STATE-OF-THE-ART VECTOR PROCESSORS

	This work Lane ₁₆ Reg ₃₂	TC'24 [†] [19]	ICCAD'22 [‡] [52]	TCASII'23 [†] [53]	TCASII'23 [38]	TVLSI'23 [54]	TVLSI'24 [29]
ISA Base & Extensions	Xuvp	GCV1.0	Zve32x	GCV0.9	IMFAC×16	IMA+Vec/Mtx	V1.0+Tensor
Technology (nm)	40	22	22	65	65	28	28
Result Source	Synthesis	Layout	Layout	Silicon	Silicon	Silicon	Synthesis
Frequency (MHz)	400	1350	594	280	205	1000	1050
Supply (V)	1.1	0.8	0.8	1.2	1.2	1.125	0.9
Core Area (mm ²)	0.94	0.95	20.1	6	10	72	1.2
Int. Formats (bit)	8, 16	8, 16, 32, 64	8, 16, 32	8, 16, 32, 64	2, 4, 8, 16, 32	8	4, 8, 16, 32, 64
Best Int. Perf. (GOPS@INT8)	19.9	83.5	285	22.9	58@INT2	13000	737.9@INT4
Best Int. Area Eff. (GOPS/mm ²)	21.2	87.9	14.2	3.8	5.8@INT2	180.6	614.6@INT4
Best Int. Energy Eff. (GOPS/W)	250.5	376.0	266	100.5	1152@INT2	630	1383.4@INT4
Normalized to 40nm technology*							
Norm. Area (mm ²)	0.94	3.1	66.4	2.3	3.8	146.9	2.4
Norm. Int. Perf. (GOPS@INT8)	19.9	45.9	156.8	37.2	94.3@INT2	9100	516.5@INT4
Norm. Int. Area Eff. (GOPS/mm ²)	21.2	14.8	2.4	16.2	24.8@INT2	61.9	215.2@INT4

[†] A 4-lane configuration.

[‡] A Mempoool₆₄Spatz₄ configuration.

* All results are normalized to 40nm based on: frequency $\propto s$, area $\propto s^{-2}$, where s is the scaling factor.

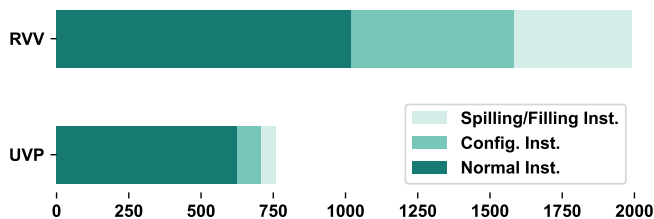


Fig. 8. Fraction of instructions over the dynamic instruction count for the 2048-point FFT kernel.

B. Instruction Breakdown

To further assess the impact of UVP, we analyze the dynamic instruction count of both RVV and UVP implementations. SystemVerilog Assertions (SVA) are integrated at the RTL level to track register spilling/filling and strip-mining-related instructions. As shown in Fig. 8, UVP reduces the dynamic instruction count for the 2048-point FFT kernel to 38% of that observed in RVV, underscoring three notable improvements in the proposed design. First, UVP reduces arithmetic instruction count by 1.6 $\times$, attributed to the introduction of dedicated complex multiplication instructions, which streamline computation. Second, configuration instructions — such as `vsetvl` in RVV and `uvp_vsetcsr` in UVP — are significantly reduced due to UVP’s flexible register grouping mechanism, which accommodates longer vectors and minimizes status switching overhead. Finally, UVP considerably decreases register spilling and filling instructions, owing to its asymmetric gather/scatter instructions and the kernel design that departs from the conventional divide-and-conquer approach, further enhancing computational efficiency.

C. Implementation Result

We implement our proposed architecture at the RTL level and synthesize the design using Synopsys Design Compiler

on the SMIC 40nm technology. Throughput performance is evaluated by issuing randomly generated instructions (with varied RGs and AVLs) to the extension using Synopsys VCS. Timing and power analyses are carried out under typical process, voltage, and temperature conditions (TT, 1.1V, 25°C). We label each configuration of our work as Lane _{l} Reg _{r} , where l denotes the number of lanes and r represents the depth of SRAM.

1) *Comparison with the State-of-the-Arts:* Table III presents a comprehensive performance comparison with the state-of-the-art vector processors, normalizing performance and efficiency across different technologies for consistency. The proposed architecture is compared with the synthesis results of other processors to ensure a fair evaluation of computational capability. In Ara2 [19] and Yun [53], lane-based vector processors supporting ratified RVV extensions with high-performance scalar cores are implemented and fabricated. Both Spatz [52] and Dustin [38] address the Von Neumann bottleneck by reducing the impact of instruction fetching in multi-core architectures but at different scales: the former integrates a tightly coupled vector processing unit within each lightweight core, while the latter synchronizes all cores in lockstep by disabling individual instruction fetch units. To accommodate the intensive demands of AI workloads, HIPU [54] and SPEED [29] introduce dedicated matrix/tensor computation units.

UVP falls short of achieving the maximum frequency due to two primary factors. First, each lane incorporates densely packed arithmetic units which introduce high fan-in logic, often implemented using dense and-or-invert (AOI) cells. This leads to significant local routing congestion. Second, the centralized EXE must maintain datapath connectivity with all surrounding lanes, resulting in placement contention and increased routing complexity.

To align with the execution unit configurations of Ara2

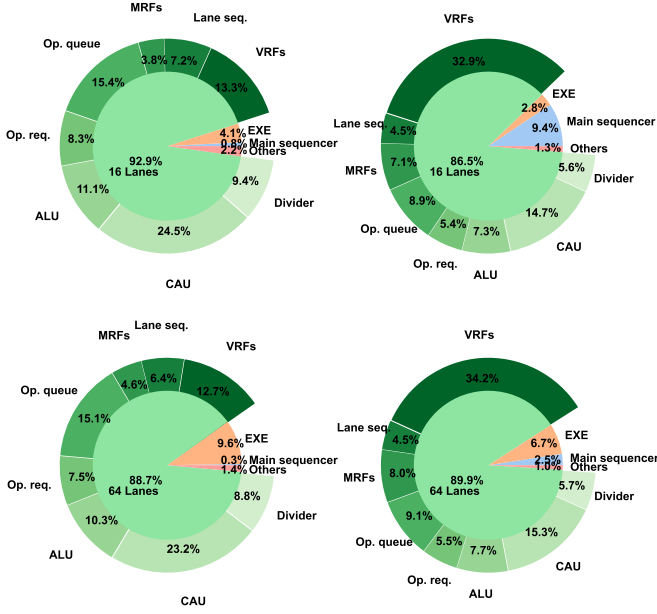


Fig. 9. Area breakdown of vector extension under different configurations. Upper left: Lane₁₆Reg₃₂; Upper right: Lane₁₆Reg₁₀₂₄; Bottom left: Lane₆₄Reg₃₂; Bottom right: Lane₆₄Reg₁₀₂₄. The operand requester (*Op. req.*) and operand queue (*Op. queue*) within each lane handle operand fetching and temporary storage prior to the execution, respectively.

[19] and Yun [53], we employ a 16-lane setup with 32 vector registers for comparison. In terms of throughput, Ara2 [19] and Yun [53] achieve at least $1.87\times$ speedup in peak throughput under normalized conditions compared to our work. This advantage is likely due to their higher normalized frequency and better utilization across all execution units. However, our extension demonstrates superior area efficiency, outperforming these lane-based designs by 43.2% and 30.9%, respectively. This improvement arises from focusing on integer arithmetic computation and vector permutation while eliminating floating-point units, resulting in a more compact core area.

Our design is further compared with other micro-architectures. While Dustin [38] achieves peak throughput with 2-bit quantization, our extension potentially surpasses multi-core architectures in terms of area and efficiency. This outcome stems from the fact that, despite efforts to equip scalar cores with vector capabilities, some datapaths within the pipeline remain idle during vector processing, resulting in significant silicon waste. However, a substantial gap exists between our design and AI-focused extensions, namely, SPEED [29] and HIPU [54]. The high throughput of these architectures is driven by the numerous PEs in their systolic arrays, optimized for tensor computations, and the minimal control overhead. Although the addition of execution units increases silicon area, the resulting area efficiencies still outperform our design due to the overwhelming throughput achieved by concurrently operating PEs, which effectively amortizes the silicon overhead. Nonetheless, it is worth noting that both SPEED and HIPU are specifically tailored for tensor computing in AI workloads and lack evaluation results in the domain of wireless signal processing.

2) *Area Breakdown*: The area breakdown of the proposed extension across different configurations is illustrated in Fig.

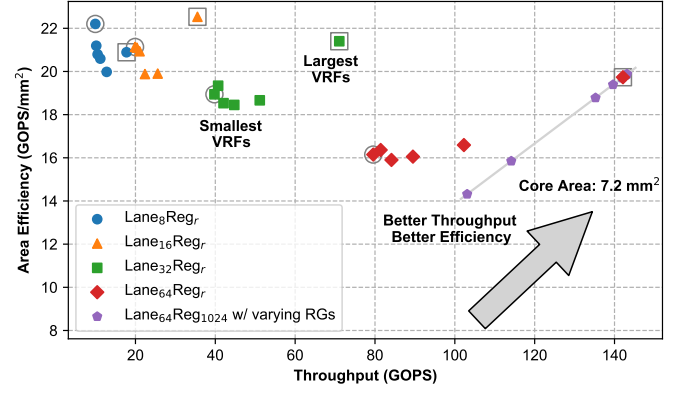


Fig. 10. Comparison of throughput and area efficiency under varying VRF and RG configurations. (*r*: Number of vector registers)

9. Generally, the lanes occupy the largest area proportion, with minimal overhead from the main sequencer and EXE. When *r* is small, the main sequencer, which includes the proposed UVP detection, has a negligible area share. However, as *r* increases, the area contribution of the sequencer grows due to additional compare logic. Additionally, the number of PEs in EXE scales with *l*, resulting in a larger EXE area as the lane count increases.

The lane breakdown is depicted around the internal pie chart, revealing how the SRAM footprint significantly impacts the distribution. When *r* is small, the computation logic – including the ALU, CAU, and divider – accounts for roughly 45% of the lane area, while SRAMs contribute around 13%. However, as *r* increases, the computation logic proportion drops to 28%, a shift that partly explains the decreased area efficiency at higher *r* values. This insight suggests that adding more execution units within each lane could help improve area efficiency.

3) *Design Space Exploration*: To further assess the benefit of UVP, we examine register spilling and filling impacts. Our extension is tested with various lane counts and VRF SRAM configurations, with *l* ranging from 8 to 64 and *r* from 32 to 1024. Additionally, we set AVL to the maximum value *r* = 1024 can support without spilling. Fig. 10 illustrates that as lane count rises, throughput increases, though area efficiency declines slightly due to the significant non-computation logic discussed in Section V-C2. In the first four scatter plots, throughput generally improves as *r* increases, with area efficiency remaining steady. Excluding *r* = 1024, there is a $1.27\times$ speedup when comparing the smallest VRFs (gray circles, *r* = 32) and configurations with one spill (*r* = 512). The largest VRFs (gray squares) deliver optimal throughput by avoiding register spilling and filling, achieving a $1.79\times$ speedup.

To showcase the advantages of the large RGs, an additional evaluation is conducted with a fixed configuration, Lane₆₄Reg₁₀₂₄, featuring a core area of 7.2 mm². The purple scatter plots in Fig. 10 illustrate the throughput increase as the size of the RG grows. Due to the reduced likelihood of data hazards and read/write conflicts, the maximum throughput is $1.38\times$ higher than that of the *r* = 32 configuration.

VI. CONCLUSION AND FUTURE WORK

We introduce Unlimited Vector Processing (UVP), a methodology that enables efficient and flexible vector processing for wireless baseband processing. UVP presents a novel architecture that supports non-power-of-two register grouping and hardware strip-mining. A programming model is proposed to seamlessly handle vectors of arbitrary lengths, significantly reducing strip-mining overhead and improving overall throughput. Additionally, UVP categorizes vector instructions into symmetric and asymmetric types, employing tailored load/store strategies that optimize execution across a wide range of workloads. The hardware implementation features hazard detection logic and optimized data pipelines, ensuring efficient execution of complex operations, particularly in fixed-point computations.

The experimental results demonstrate the effectiveness of UVP, achieving remarkable performance improvements with speedups of up to 3.0 $\times$ in matrix multiplication and 2.1 $\times$ in FFT kernels compared to traditional lane-based vector architectures. The synthesized RTL for a 16-lane configuration under SMIC 40nm technology occupies only 0.94 mm², delivering an impressive area efficiency of 21.2 GOPS/mm². Future work will focus on enhancing overall throughput by optimizing the EXE micro-architecture and incorporating multiple EXEs within the extension to mitigate pipeline stalling caused by limited execution units.

REFERENCES

- [1] L. Oliker, A. Canning, J. Carter, J. Shalf, and S. Ethier, "Scientific computations on modern parallel vector systems," in *SC'04: Proceedings of the 2004 ACM/IEEE Conference on Supercomputing*, pp. 10–10, IEEE, 2004.
- [2] L. Ferreira, S. Malkowsky, P. Persson, S. Karlsson, K. Åström, and L. Liu, "Design of an application-specific VLIW vector processor for ORB feature extraction," *Journal of Signal Processing Systems*, vol. 95, no. 7, pp. 863–875, 2023.
- [3] N. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, *et al.*, "Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pp. 1–14, 2023.
- [4] RISC-V, "riscv-v-spec," 2021. [Online]. Available: <https://github.com/riscvarchive/riscv-v-spec>.
- [5] P. Nannipieri, S. Di Matteo, L. Zulberti, F. Albicocchi, S. Saponara, and L. Fanucci, "A RISC-V post quantum cryptography instruction set extension for number theoretic transform to speed-up CRYSTALS algorithms," *IEEE Access*, vol. 9, pp. 150798–150808, 2021.
- [6] Y.-M. Kuo, F. García-Herrero, O. Ruano, and J. A. Maestro, "RISC-V galois field ISA extension for non-binary error-correction codes and classical and post-quantum cryptography," *IEEE Transactions on Computers*, vol. 72, no. 3, pp. 682–692, 2022.
- [7] M. Gautschi, P. D. Schiavone, A. Traber, I. Loi, A. Pullini, D. Rossi, E. Flamand, F. K. Gürkaynak, and L. Benini, "Near-threshold RISC-V core with DSP extensions for scalable iot endpoint devices," *IEEE transactions on very large scale integration (VLSI) systems*, vol. 25, no. 10, pp. 2700–2713, 2017.
- [8] L. Jiang, Y. Shi, Y. Liu, Q. Deng, S. Xu, Y. Shen, F. Ye, S. Cao, and Z. Jiang, "A hierarchical dataflow-driven heterogeneous architecture for wireless baseband processing," in *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, pp. 587–593, 2025.
- [9] Q. Spencer, A. Swindlehurst, and M. Haardt, "Zero-forcing methods for downlink spatial multiplexing in multiuser mimo channels," *IEEE Transactions on Signal Processing*, vol. 52, no. 2, pp. 461–471, 2004.
- [10] F. Tosato and P. Bisaglia, "Simplified soft-output demapper for binary interleaved COFDM with application to HIPERLAN/2," in *2002 IEEE International Conference on Communications. Conference Proceedings. ICC 2002 (Cat. No. 02CH37333)*, vol. 2, pp. 664–668, IEEE, 2002.
- [11] K. C. Behera, "An efficient low-latency algorithm and implementation for rate-matching and bit-interleaving in 5G NR," in *2020 IEEE 3rd 5G World Forum (5GWF)*, pp. 565–571, IEEE, 2020.
- [12] J. Janhunen, T. Pitkanen, O. Silven, and M. Juntti, "Fixed-and floating-point processor comparison for mimo-ofdm detector," *IEEE Journal of Selected Topics in Signal Processing*, vol. 5, no. 8, pp. 1588–1598, 2011.
- [13] C. Chen, X. Xiang, C. Liu, Y. Shang, R. Guo, D. Liu, Y. Lu, Z. Hao, J. Luo, Z. Chen, *et al.*, "Xuantie-910: A commercial multi-core 12-stage pipeline out-of-order 64-bit high performance RISC-V processor with vector extension: Industrial product," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pp. 52–64, IEEE, 2020.
- [14] SiFive, "SiFive intelligence X280," 2022. [Online]. Available: https://sifive.cdn.prismic.io/sifive/70445cba-0549-475e-a538-5c09a402efbc_x280-datasheet-22G1.pdf.
- [15] Andes technology, "AndesCore™ NX27V Processor," 2021. [Online]. Available: https://www.andestech.com/wp-content/uploads/AndesCore_NX27V_Product_Package_PB156_V1.2.pdf.
- [16] Semidynamics, "Semidynamics Vector Unit - Only 100% customisable RISC-V Vector Unit," 2024. [Online]. Available: <https://semidynamics.com/en/technology/vector-unit>.
- [17] M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, and L. Benini, "Ara: A 1-GHz+ scalable and energy-efficient RISC-V vector processor with multiprecision floating-point support in 22-nm FD-SOI," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 2, pp. 530–543, 2019.
- [18] M. Perotti, M. Cavalcante, N. Wistoff, R. Andri, L. Cavigelli, and L. Benini, "A 'New Ara' for vector computing: An open source highly efficient RISC-V V 1.0 vector processor design," in *2022 IEEE 33rd International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, pp. 43–51, IEEE, 2022.
- [19] M. Perotti, M. Cavalcante, R. Andri, L. Cavigelli, and L. Benini, "Ara2: Exploring single-and multi-core vector processing with an efficient RVV 1.0 compliant open-source processor," *IEEE Transactions on Computers*, 2024.
- [20] E. Humblet, T. Dupuis, Y. Fournier, M. H. AskariHemmat, F. Leduc-Primeau, J. P. David, and Y. Savaria, "MSPARQ: A RISC-V vector processor array optimized for low-resolution neural networks," in *2024 IEEE 67th International Midwest Symposium on Circuits and Systems (MWSCAS)*, pp. 464–468, IEEE, 2024.
- [21] Y. Wang, M. Yang, C.-P. Lo, and J. P. Kulkarni, "30.6 Vecim: A 289.13 GOPS/W RISC-V vector co-processor with compute-in-memory vector register file for efficient high-performance computing," in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 67, pp. 492–494, IEEE, 2024.
- [22] F. Minervini, O. Palomar, O. Unsal, E. Reggiani, J. Quiroga, J. Marimon, C. Rojas, R. Figueras, A. Ruiz, A. Gonzalez, *et al.*, "Vitruvius+: An area-efficient RISC-V decoupled vector coprocessor for high performance computing applications," *ACM Transactions on Architecture and Code Optimization*, vol. 20, no. 2, pp. 1–25, 2023.
- [23] M. Platzer and P. Puschner, "Vicuna: A timing-predictable RISC-V vector coprocessor for scalable parallel computation," in *33rd euromicro conference on real-time systems (ECRTS 2021)*, Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2021.
- [24] I. A. Assir, M. E. Iskandarani, H. R. A. Sandid, and M. A. Saghir, "Arrow: A RISC-V vector accelerator for machine learning inference," *arXiv preprint arXiv:2107.07169*, 2021.
- [25] Y. Lee, A. Waterman, R. Avizienis, H. Cook, C. Sun, V. Stojanović, and K. Asanović, "A 45nm 1.3 GHz 16.7 double-precision GFLOPS/W RISC-V processor with vector accelerators," in *ESSCIRC 2014-40th European Solid State Circuits Conference (ESSCIRC)*, pp. 199–202, IEEE, 2014.
- [26] C. Schmidt, J. Wright, Z. Wang, E. Chang, A. Ou, W. Bae, S. Huang, V. Milovanović, A. Flynn, B. Richards, *et al.*, "An eight-core 1.44-GHz RISC-V vector processor in 16-nm FinFET," *IEEE Journal of Solid-State Circuits*, vol. 57, no. 1, pp. 140–152, 2021.
- [27] M. Johns and T. J. Kazmierski, "A minimal RISC-V vector processor for embedded systems," in *2020 Forum for Specification and Design Languages (FDL)*, pp. 1–4, IEEE, 2020.
- [28] G. Ottavi, A. Garofalo, G. Tagliavini, F. Conti, L. Benini, and D. Rossi, "A mixed-precision RISC-V processor for extreme-edge DNN inference," in *2020 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pp. 512–517, IEEE, 2020.
- [29] C. Wang, C. Fang, X. Wu, Z. Wang, and J. Lin, "SPEED: A scalable RISC-V vector processor enabling efficient multiprecision DNN inference," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2024.

- [30] K. Li, J. Zhou, Y. Wang, J. Luo, Z. Yang, S. Yang, W. Mao, M. Huang, and H. Yu, "A precision-scalable energy-efficient bit-split-and-combination vector systolic accelerator for NAS-optimized DNNs on edge," in *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 730–735, IEEE, 2022.
- [31] M. Attari, L. Ferreira, L. Liu, and S. Malkowsky, "An application specific vector processor for efficient massive MIMO processing," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 9, pp. 3804–3815, 2022.
- [32] K. Patsidis, C. Nicopoulos, G. C. Sirakoulis, and G. Dimitrakopoulos, "RISC-V²: a scalable RISC-V vector processor," in *2020 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, IEEE, 2020.
- [33] C. R. Lazo, E. Reggiani, C. R. Morales, R. F. Bagué, L. A. V. Vargas, M. A. R. Salinas, M. V. Cortés, O. S. Ünsal, and A. Cristal, "Adaptable register file organization for vector processors," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 786–799, IEEE, 2022.
- [34] J. M. Domingos, N. Neves, N. Roma, and P. Tomás, "Unlimited vector extension with data streaming support," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pp. 209–222, IEEE, 2021.
- [35] A. Fernandes, L. Crespo, N. Neves, P. Tomás, N. Roma, and G. Falcao, "Functional validation of the RISC-V unlimited vector extension," *IEEE Embedded Systems Letters*, 2024.
- [36] T. Ta, K. Al-Hawaj, N. Cebry, Y. Ou, E. Hall, C. Golden, and C. Batten, "Big.VLITTLE: On-demand data-parallel acceleration for mobile systems on chip," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 181–198, IEEE, 2022.
- [37] P. Bedoukian, N. Adit, E. Peguero, and A. Sampson, "Software-defined vector processing on manycore fabrics," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 392–406, 2021.
- [38] G. Ottavi, A. Garofalo, G. Tagliavini, F. Conti, A. Di Mauro, L. Benini, and D. Rossi, "Dustin: A 16-cores parallel ultra-low-power cluster with 2b-to-32b fully flexible bit-precision and vector lockstep execution mode," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 70, no. 6, pp. 2450–2463, 2023.
- [39] E. Mahurin, "Qualcomm® hexagon™ NPU," in *2023 IEEE Hot Chips 35 Symposium (HCS)*, pp. 1–19, IEEE Computer Society, 2023.
- [40] Y.-C. Lin, Y.-P. You, and J.-K. Lee, "PALF: compiler supports for irregular register files in clustered VLIW DSP processors," *Concurrency and computation: practice and experience*, vol. 19, no. 18, pp. 2391–2406, 2007.
- [41] C.-B. Kuan and J. K. Lee, "Compiler supports for VLIW DSP processors with SIMD intrinsics," *Concurrency and Computation: Practice and Experience*, vol. 24, no. 5, pp. 517–532, 2012.
- [42] M.-S. Yu, H.-C. Chang, C.-T. Wang, Y.-W. Tien, T.-L. Chen, and J.-K. Lee, "Optimizing computer vision algorithms with TVM on VLIW architecture based on RVV," *The Journal of Supercomputing*, vol. 81, no. 1, p. 172, 2025.
- [43] L. Codrescu, W. Anderson, S. Venkumhanthi, M. Zeng, E. Plondke, C. Koob, A. Ingle, C. Tabony, and R. Maule, "Hexagon DSP: An architecture optimized for mobile multimedia and communications," *IEEE Micro*, vol. 34, no. 2, pp. 34–43, 2014.
- [44] CEVA Inc., "Introducing CEVA-XC16," 2020. [Online]. Available: https://www.ceva-dsp.com/wp-content/uploads/2020/03/CEVA-XC16_introduction_non_nda_public-v2.pdf.
- [45] ARM, "Learn the architecture - Optimizing C code with Neon intrinsics," 2025. [Online]. Available: <https://developer.arm.com/documentation/102467/0201>.
- [46] Pulp-platform, "Ara Apps," 2021. [Online]. Available: <https://github.com/pulp-platform/ara/tree/main/apps>.
- [47] J. Acevedo, F. H. Fitzek, and P. Seeling, "5G channel estimation kernels on RISC-V vector digital signal processors," in *2024 International Conference on Microelectronics (ICM)*, pp. 1–8, IEEE, 2024.
- [48] S. Sengupta, M. Harris, Y. Zhang, and J. D. Owens, "Scan Primitives for GPU Computing," in *SIGGRAPH/Eurographics Workshop on Graphics Hardware* (M. Segal and T. Aila, eds.), 2007.
- [49] H.-M. Lai and J.-K. Lee, "Efficient support of the scan vector model for RISC-V vector extension," in *Workshop Proceedings of the 51st International Conference on Parallel Processing*, pp. 1–8, 2022.
- [50] C.-H. Chang, C.-C. Yang, J.-K. Lee, and Y.-C. Lin, "Case study: Support OpenCL complex class for baseband computing," in *Proceedings of the International Workshop on OpenCL*, pp. 1–2, 2019.
- [51] M. Bertuletti, Y. Zhang, A. Vanelli-Coralli, and L. Benini, "Efficient parallelization of 5G-PUSCH on a scalable RISC-V many-core processor," in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 1–6, IEEE, 2023.
- [52] M. Cavalcante, D. Wüthrich, M. Perotti, S. Riedel, and L. Benini, "Spatz: A compact vector processing unit for high-performance and energy-efficient shared-L1 clusters," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, pp. 1–9, 2022.
- [53] M. Perotti, M. Cavalcante, A. Ottaviano, J. Liu, and L. Benini, "Yun: An open-source, 64-bit RISC-V-based vector processor with multi-precision integer and floating-point support in 65-nm CMOS," *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2023.
- [54] W. Zhao, G. Yang, T. Xia, F. Chen, N. Zheng, and P. Ren, "HIPU: A hybrid intelligent processing unit with fine-grained ISA for real-time deep neural network inference applications," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 12, pp. 1980–1993, 2023.